

Scikit-Image Functions

1. hog() — Histogram of Oriented Gradients

Source: from skimage.feature import hog

hog(image, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(3, 3), block_norm=None, visualize=False, visualise=None, transform_sqrt=False, feature_vector=True, multichannel=None)

Attribute	Type	Default	Options / Meaning
image	ndarray	required	Input image (M, N) or (M, N, C)
orientations	int	9	Number of orientation bins
pixels_per_cell	2-tuple (int, int)	(8, 8)	Size of each cell in pixels
cells_per_block	2-tuple (int, int)	(3, 3)	Number of cells per block
block_norm	str or None	None	'L1', 'L1-sqrt', 'L2', 'L2-Hys'
visualize	bool	False	If True, also return HOG image
visualise	bool	None	Deprecated alias for visualize
transform_sqrt	bool	False	Apply power law compression

feature_vector	bool	True	If True, return flattened feature vector
multichannel	bool or None	None	If True, treat last axis as channels

Block normalization options (detailed):

- 'L1': Normalization using L1-norm (default in some versions)
- 'L1-sqrt': L1-norm followed by square root
- 'L2': Normalization using L2-norm
- 'L2-Hys': L2-norm, limit values to 0.2, renormalize with L2-norm

Code usage in manual:

```
features, hog_image = hog(image, pixels_per_cell=(8,8),  
                           cells_per_block=(2,2), visualize=True)
```

Exam Focus: Know orientations, pixels_per_cell, cells_per_block, and block_norm options.

2. local_binary_pattern() — LBP

Source: from skimage.feature import local_binary_pattern

local_binary_pattern(image, P, R, method='default')

Attribute	Type	Default	Options / Meaning
image	(M, N) array	required	2D grayscale image
P	int	required	Number of circularly symmetric neighbor points
R	float	required	Radius of circle (spatial resolution)
method	str	'default'	See options below

Method options:

- 'default': Original LBP — grayscale invariant but NOT rotation invariant
- 'ror': Rotation-invariant extension of default

- 'uniform': Uniform patterns — grayscale invariant AND rotation invariant, finer angular quantization
- 'nri_uniform': Non-rotation-invariant uniform patterns — only grayscale invariant
- 'var': Rotation-invariant variance measures of contrast — rotation but NOT grayscale invariant

Code usage in manual:

lbp_image = feature.local_binary_pattern(image, n_points, radius, method='uniform')

Exam Focus: Know all 5 method options. 'uniform' is most commonly used.

OpenCV Functions

3. cv2.filter2D() — Convolution / Filtering

cv2.filter2D(src, ddepth, kernel[, dst[, anchor[, delta[, borderType]]]])

Attribute	Type	Default	Options / Meaning
src	InputArray	required	Input image
ddepth	int	required	Output image depth; use -1 for same as input
kernel	InputArray	required	Convolution kernel (filter matrix)
dst	OutputArray	optional	Output image
anchor	Point	(-1, -1)	Anchor point; default means kernel center
delta	double	0	Value added to results before storing
borderType	int	BORDER_DEFAULT	Pixel extrapolation method

BorderType options:

- BORDER_CONSTANT — pad with a constant value
- BORDER_REPLICATE — replicate edge pixels
- BORDER_REFLECT — reflect pixels (mirror with edge repeated)
- BORDER_REFLECT_101 — reflect pixels (mirror without edge repeated) — this is BORDER_DEFAULT
- BORDER_WRAP — wrap around
- BORDER_TRANSPARENT — transparent border
- BORDER_ISOLATED — do not look outside image

ddepth options (common):

- -1 — same as source
- cv2.CV_8U, cv2.CV_16U, cv2.CV_16S, cv2.CV_32F, cv2.CV_64F

Code usages in manual:

blurred_image = cv2.filter2D(image, -1, kernel)

energy = cv2.filter2D(image2, -1, np.ones((3,3)))**

contrast = cv2.filter2D(image, -1, np.ones((3,3)))

Exam Focus: Know ddepth=-1, anchor=(-1,-1), and the common borderType options.

4. cv2.Sobel() — Sobel Edge Detection

cv2.Sobel(src, ddepth, dx, dy[, dst[, ksize[, scale[, delta[, borderType]]]])

Attribute	Type	Default	Options / Meaning
src	InputArray	required	Input image
ddepth	int	required	Output image depth
dx	int	required	Order of derivative in x direction
dy	int	required	Order of derivative in y direction
dst	OutputArray	optional	Output image

ksize	int	3	Size of Sobel kernel; must be 1, 3, 5, or 7
scale	double	1	Optional scale factor for derivative values
delta	double	0	Optional delta added before storing
borderType	int	BORDER_DEFAULT	Pixel extrapolation method

ksize options: 1, 3, 5, 7

- If ksize = -1, a 3×3 Scharr filter is used (better than 3×3 Sobel)

Code usages in manual:

gradient_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3) # x-direction

gradient_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3) # y-direction

Exam Focus: Know that (dx=1, dy=0) = horizontal gradient, (dx=0, dy=1) = vertical gradient. Use cv2.CV_64F to avoid truncation.

5. cv2.Scharr() — Scharr Edge Detection

cv2.Scharr(src, ddepth, dx, dy[, dst[, scale[, delta[, borderType]]]])

Attribute	Type	Default	Options / Meaning
src	InputArray	required	Input image
ddepth	int	required	Output image depth
dx	int	required	Order of derivative in x direction
dy	int	required	Order of derivative in y direction

dst	OutputArray	optional	Output image
scale	double	1	Optional scale factor
delta	double	0	Optional delta value
borderType	int	BORDER_DEFAULT	Pixel extrapolation method

Note: Scharr has no ksize parameter — it always uses a fixed 3×3 kernel.
Code usage in manual:

```
scharr_x = cv2.Scharr(image, cv2.CV_64F, 1, 0)
```

```
scharr_y = cv2.Scharr(image, cv2.CV_64F, 0, 1)
```

Exam Focus: Scharr is more accurate than 3×3 Sobel but only works with 3×3 kernel size.

6. cv2.GaussianBlur() — Gaussian Smoothing

cv2.GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]])

Attribute	Type	Default	Options / Meaning
src	InputArray	required	Input image
ksize	Size (width, height)	required	Gaussian kernel size; both must be positive and odd, or zeros
sigmaX	double	required	Gaussian standard deviation in X direction
dst	OutputArray	optional	Output image
sigmaY	double	0	Standard deviation in Y; if 0, set equal to sigmaX

borderType	int	BORDER_D EFAULT	Pixel extrapolation method
------------	-----	--------------------	-------------------------------

ksize options:

- Must be odd: (3,3), (5,5), (7,7), etc.
- Or (0,0) — then computed from sigma

Code usage in manual:

image_smoothed = cv2.GaussianBlur(image, (5, 5), 0)

Exam Focus: ksize must be odd. If sigmaY=0, it equals sigmaX.

7. cv2.getGaussianKernel() — Generate Gaussian Kernel

cv2.getGaussianKernel(ksize, sigma[, ktype])

Attribute	Type	Default	Options / Meaning
ksize	int	required	Aperture size; must be odd and positive
sigma	double	required	Gaussian standard deviation; if non-positive, computed from ksize
ktype	int	CV_64F	Type of filter coefficients: CV_32F or CV_64F

Code usage in manual:

kernel = cv2.getGaussianKernel(kernel_size[0], sigma)

Exam Focus: Returns a 1D kernel. For 2D, you multiply the x and y kernels.

8. cv2.Canny() — Canny Edge Detector

cv2.Canny(image, threshold1, threshold2[, edges[, apertureSize[, L2gradient]]])

Attribute	Type	Default	Options / Meaning
-----------	------	---------	-------------------

image	InputArray	required	8-bit input image
threshold1	double	required	Low threshold for hysteresis
threshold2	double	required	High threshold for hysteresis
edges	OutputArray	optional	Output edge map (single-channel 8-bit)
apertureSize	int	3	Aperture size for Sobel operator
L2gradient	bool	False	If True, use L2 norm; if False, use L1 norm

apertureSize options: 3, 5, 7

L2gradient options:

- True — more accurate L2 norm:
 - $(dl/dx)^2 + (dl/dy)^2$
 - (dl/dx)
 - 2
 - $+(dl/dy)$
 - 2
 -
- False — default L1 norm:
 - $|dl/dx| + |dl/dy|$
 - $|dl/dx| + |dl/dy|$

Code usage in manual:

edges = cv2.Canny(image, threshold1=100, threshold2=200)

edges = cv2.Canny(image_smoothed, threshold1=30, threshold2=70)

Exam Focus: threshold1 = low threshold, threshold2 = high threshold. Higher thresholds → fewer edges, less noise.

NumPy Functions

9. np.histogram() — Histogram Computation

np.histogram(a, bins=10, range=None, density=None, weights=None)

Attribute	Type	Default	Options / Meaning
a	array_like	required	Input data (flattened array)
bins	int or sequence	10	Number of bins or bin edges
range	tuple (float, float)	None	Lower and upper range of bins
density	bool	None	If True, returns probability density
weights	array_like	None	Weights for each value

Code usages in manual:

```
hist, bin_edges = np.histogram(gradient_orientation, bins=8, range=(0, 360))
hist, _ = np.histogram(lbp_image.ravel(), bins=np.arange(0, n_points + 3),
                      range=(0, n_points + 2))
energy_histogram, energy_bins = np.histogram(energy, bins=256, range=(0,
energy.max()))
contrast_histogram, contrast_bins = np.histogram(contrast, bins=256,
                                                range=(0, contrast.max()))
```

Exam Focus: Returns (histogram, bin_edges). Use .ravel() to flatten 2D arrays.

10. np.arctan2() — Gradient Orientation

np.arctan2(y, x)

Attribute	Type	Meaning
y	array_like	y-coordinate (gradient_y)

x	array_like	x-coordinate (gradient_x)
---	------------	---------------------------

Code usage in manual:

gradient_orientation = np.arctan2(gradient_y, gradient_x) * 180 / np.pi

Exam Focus: Returns angle in radians from $-\pi$ to π . Multiply by $180/\pi$ for degrees.

11. np.sqrt() — Square Root

np.sqrt(x)

Code usage in manual:

gradient_magnitude = np.sqrt(gradient_x2 + gradient_y**2)**

Matplotlib Functions (Visualization)

12. plt.imshow() — Display Image

Attribute	Options
cmap	'gray', 'viridis', 'jet', 'hot', etc.

Code usage in manual:

plt.imshow(image, cmap=plt.cm.gray)

plt.imshow(lbp_image, cmap='gray')

13. plt.bar() — Bar Chart

Attribute	Options
width	float (e.g., 45)
align	'center', 'edge'

color	'b', 'r', 'g', etc.
-------	---------------------

Code usage in manual:

```
plt.bar(range(0, n_points + 2), hist)
plt.bar(bin_edges[:-1], hist, width=45, align='center')
```

14. plt.subplot() — Create Subplot

Signature: plt.subplot(nrows, ncols, index)

Code usage in manual:

```
plt.subplot(121) # 1 row, 2 columns, first plot
plt.subplot(122) # 1 row, 2 columns, second plot
```

Summary Table for Quick Revision

Function	Key Attributes	Key Options
hog()	orientations, pixels_per_cell, cells_per_block, block_norm	'L1', 'L1-sqrt', 'L2', 'L2-Hys'
local_binary_pattern()	P, R, method	'default', 'ror', 'uniform', 'nri_uniform', 'var'
cv2.filter2D()	ddepth, kernel, anchor, delta, borderType	BORDER_CONSTANT, BORDER_REFLECT, BORDER_DEFAULT, etc.
cv2.Sobel()	ddepth, dx, dy, ksize	ksize: 1, 3, 5, 7, or -1 (Scharr)
cv2.Scharr()	ddepth, dx, dy	No ksize (fixed 3×3)
cv2.GaussianBlur()	ksize, sigmaX, sigmaY	ksize must be odd

<code>cv2.getGaussianKernel()</code>	<code>ksize, sigma, ktype</code>	<code>CV_32F, CV_64F</code>
<code>cv2.Canny()</code>	<code>threshold1, threshold2, apertureSize, L2gradient</code>	<code>apertureSize: 3, 5, 7; L2gradient: True/False</code>
<code>np.histogram()</code>	<code>bins, range, density</code>	Any int or sequence for bins

Exam Focus: For each function, know the required attributes (those without defaults) and the most common options used in the manual. The Canny thresholds and Sobel ksize options are the most likely to be asked.